

Turning Written Policy into Rules That Run

And catching the ones that quietly go out of date

Nitin Chaube | Cotiviti Intern Assessment | Topic 3: Content Management in Health Care | August 2026

Background

A health plan cannot pay a claim just because someone sent it in. Every claim is checked against rules that say what the plan covers. Those rules do not start in a database. They start as writing: Medicare coverage documents, national coding standards, the plan's own medical policy, guidelines from medical societies, and contracts with hospitals.

Content management here means turning that writing into something a computer can run, then keeping the two in step. That covers four jobs. Summarising a document so a reviewer can skim it, comparing two versions to see what moved, converting the requirements into rules, and looking after the result once the source starts changing.

The converting is done by hand today. A nurse or a certified coder reads the document and writes the rule. It is slow, and two people will not always write the same rule from the same paragraph. The bigger problem is that it only happens once. When the document is revised, nothing goes back to check the rules that came out of it. A rule written correctly in January can be wrong by July and carry on running, turning down claims the policy now covers, until somebody happens to spot it.

Trends

Policy changes faster than anyone can follow by reading. CMS looks after thousands of coverage documents and edits them constantly. Local coverage determinations ship with a built-in revision history because revising them is routine. Add state Medicaid rules, commercial medical policy and the yearly code updates, and no company is re-reading all of that against its own rule library.

Turning documents into structured rules has recently become practical, and vendors have noticed. Cohere Health sells a product that converts policy documents into decision trees and runnable rules, and academic work is heading the same way (Han et al., 2026; Wang et al., 2026). What strikes me is that all of it stops in the same place. Document in, rule out, tooling finishes. Nobody is still watching the document after the rule ships.

Two other pressures matter. Payers are moving from clawing money back after payment to getting the decision right beforehand, which raises the cost of a wrong rule: a recovery request can be withdrawn, but a wrong denial stops a patient getting equipment and starts an appeal. And how often a payer annoys providers is now something buyers compare. Cotiviti advertises that 97 to 99 percent of claims pass straight through and only about 10 percent of providers are touched by edits (Cotiviti, 2026a).

Opportunities

The biggest short-term opening is not writing new rules. It is looking after the ones already running. Any company with a large rule set built from documents that keep changing has out-of-date rules in

production today and no organised way to find them. A service that watches the source documents and points at the rules a revision has broken adds something without replacing anything, and it fails safely. The worst case is a reviewer looking at a finding and deciding it does not matter.

Three smaller things come out of the same machinery. Draft rules can be produced for a reviewer to approve, which cuts the time each one takes. Recording which document version and which sentence a rule came from turns defending an appeal into a lookup. And running the check backwards, looking for requirements no rule covers, finds money going out of the door rather than only preventing mistakes.

Risks

A wrong automated rule is worse than no rule. A false positive turns down a valid claim, starts an appeal, costs staff time on both sides and damages the relationship with the provider. Anything built here has to be judged on false positives first and coverage second.

Models also drop conditions and invent quotes. My own extractor produced a rule that quietly left out one of the ways a patient could qualify, which would have turned down every insulin pump user. A check caught it. It did not look wrong.

The law is tightening as well. Several states now say AI cannot be the only basis for a coverage decision and that a licensed person has to be involved, so any design that puts a model inside the decision invites trouble. Two smaller risks: code sets are licensed, so CPT descriptions cannot be copied into a model or passed to a subcontractor, and competitors already sell policy-to-rules conversion, so the model itself is not the thing worth owning.

Recommendations

Start by watching the rules that already exist. Not a writing tool. A service that follows the source documents and hands back a ranked list of rules a revision has broken, with a dollar figure attached by re-running recent claims through the old rule and the corrected one. It sells as reducing risk, it changes nothing about how rules get written today, and the dollar figure is what makes a reviewer act.

Treat rules like code, and keep the model out of the decision. Version rules and never edit them in place. Keep an index from a document version to every rule built from it. Make decisions reproducible for the date they were made, so a denial from three years ago can be replayed against the policy in force at the time. Run new rules silently against real traffic first, then on a small slice, before they affect payment. Let a model draft, a person approve, and ordinary code decide. That is what keeps the system defensible to a regulator or an appeals board, and it satisfies the state laws above without being designed around them.

Invest in the vocabulary, not the model. The part worth owning is the structured list of claim facts that policy requirements are written against, per clinical area, mapped onto real claim data. Models get swapped every few months. A carefully built vocabulary tied to a live rule library does not. In my prototype the pipeline moved between documents without changes; the vocabulary was the only piece I wrote by hand, and the only piece that limited what could be expressed.

Prototype and results

I built a working version against real CMS documents: Medicare LCD L33822, the version effective 1 January 2020 and the revision effective 18 July 2021. It reads the policy, turns it into rules that each

quote the sentence they came from, checks those quotes are real, decides sample claims, and then checks every rule against the newer version.

It found all three coverage changes CMS recorded for that revision, with no false alarms. The scoring is against the revision history CMS prints inside the document, so the answer key is not something I wrote. It also correctly reported no change across a separate pair of 2024 versions, and got there without spending a model call, because once formatting is stripped those two documents are identical character for character. Six of twenty sample claims changed decision once the out-of-date rules were fixed, worth \$1,208.71.

One result surprised me. Running the extraction five times and taking the majority answer did *not* fix the dropped-condition bug, because the model made the same mistake in most runs, so the majority was simply wrong. What fixed it was translating each rule back into plain English without showing the model where it came from, then comparing that against the original sentence. That found the missing condition with no answer key at all, and using it to settle the disagreement recovered the correct answer only one run had produced.

The tool still fails its own quality check on two rules. One needs a limit of one item per thirty days, and my claim vocabulary has no way to express a time window. I left that failure visible rather than tuning it away. It is the clearest argument for the third recommendation: the vocabulary was the bottleneck, not the model.

References

- American Diabetes Association Professional Practice Committee. (2026). 7. Diabetes technology: Standards of care in diabetes 2026. *Diabetes Care*, 49(Suppl. 1), S150–S165.
<https://doi.org/10.2337/dc26-S007>
- Centers for Medicare & Medicaid Services. (2021). *Local coverage determination: Glucose monitors (L33822), revision effective July 18, 2021*. Medicare Coverage Database.
<https://www.cms.gov/medicare-coverage-database/view/lcd.aspx?lcdid=33822>
- Centers for Medicare & Medicaid Services. (2025). *Local coverage article: Glucose monitor, policy article (A52464)*. Medicare Coverage Database. <https://www.cms.gov/medicare-coverage-database/view/article.aspx?articleId=52464>
- Centers for Medicare & Medicaid Services. (2026a). *MCD archive search*.
https://localcoverage.cms.gov/mcd_archive
- Centers for Medicare & Medicaid Services. (2026b). *Coverage API documentation*.
<https://api.coverage.cms.gov/docs/>
- Cohere Health. (2026). *Policy Studio: Medical policy management for utilization management*.
<https://www.coherehealth.com/utilization-management/policy-studio>
- Cotiviti. (2026a). *Coding validation*.
<https://www.cotiviti.com/solutions/payment-accuracy/coding-validation>
- Cotiviti. (2026b). *The payment integrity guide for health plans*. <https://resources.cotiviti.com/payment-integrity/the-health-plan-guide-to-claims-payment-integrity>
- Han, J., Liu, Y., & Chen, W. (2026). *CPGPrompt: Translating clinical guidelines into LLM-executable decision support*. arXiv. <https://arxiv.org/abs/2601.03475>